Mobile and Ubiquitous Computing

Context-awareness



FCUL, Departamento de Informática

2026/2027

Ana Paula Afonso

Context & context-aware computing



Context & context-aware computing

- **Context** is any information that can be used to characterize the *situation of an entity*

  Entity:

  person, place, or object that is considered relevant to the interaction between a user and an application

- An application/system is **context-aware** if it uses context to provide relevant information and/or services to the user, where relevance depends on the user's task

  Location-aware: presents content to users, specifically based on their geographical location

- (Dey, 2001)

Context – situational information



Context – situational information



- Conversation between humans includes more than the exchange of words
- We use multiple sources of information to increase conversational bandwidth, i.e., the information per word is greater than the words themselves
  - tone, body language, current context, previous conversations

No Context



At 12:30: I’m hungry

- What does your friend infer?
- What would a computer/phone infer?
- What extra information would your phone need?

Context & context-aware computing

- Humans share a lot of implicit situational information, but computers require explicit input. That is **where context-awareness comes in**.
  - **Context** is any information that can be used to characterize the *situation of an entity – person, place, object*

(Dey, 2001)

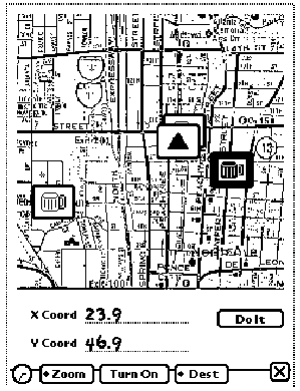
History of context-aware computing

The early **1990**'s saw the arrival of context-aware computing with the introduction of small mobile computing devices

- Olivetti Lab's Active Badge (1992)
  - infrared communication between user badges and sensors placed in a building to monitor movement of users for forwarding calls



- PARCTAB system, Xerox Parc (1993)
  - uses location information to allow applications to adapt to user's environment
- CyberGuide from Georgia Tech (1996)
  - Context-aware applications serve as tour guides by presenting information about the user's current environment
- Georgia Tech's Context Toolkit (1999)
  - Frameworks built to support context-aware applications began to be developed



Since 2000, more and more research and applications

History of context-aware computing

What other, more modern technologies or applications significantly impacted context-aware computing?

Implicit vs explicit context

- Explicit – The user tells the system directly about the context
- Implicit – System determines context automatically

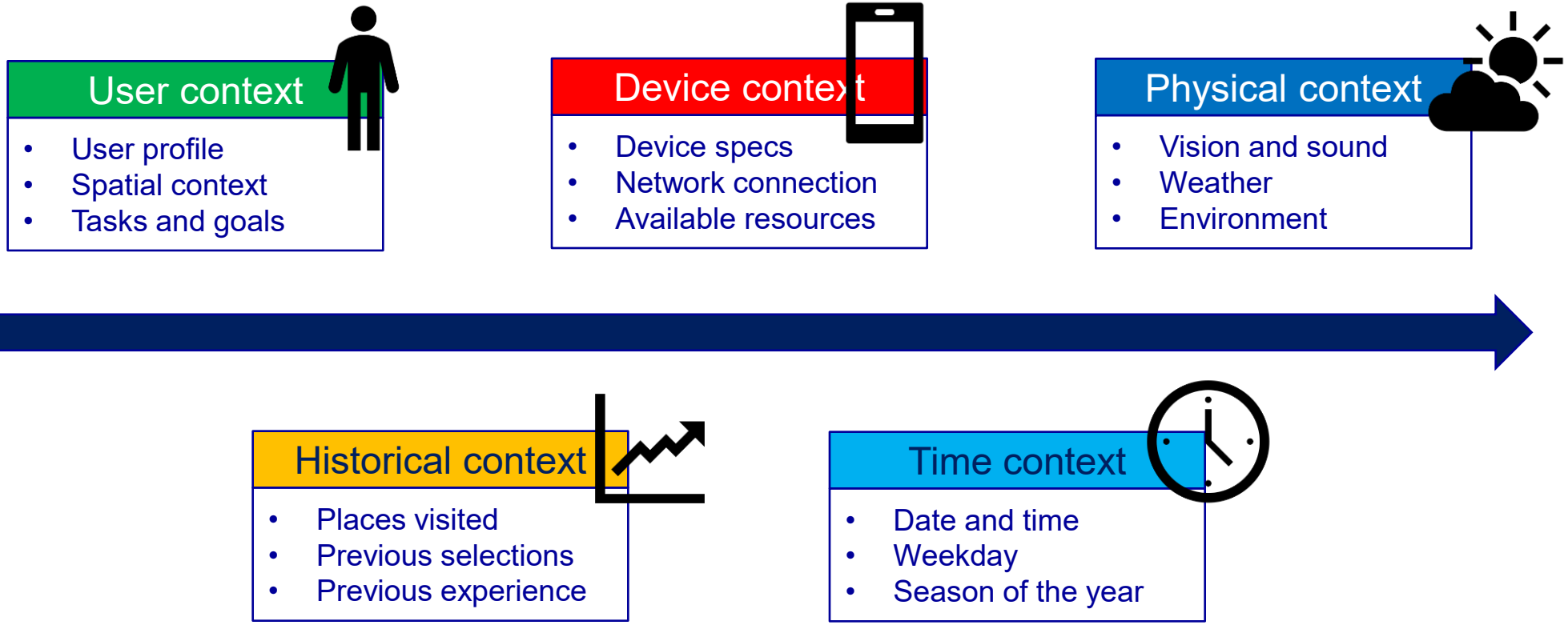
- Example: User identity
  - Face recognition – implicit
  - Type in username/password – explicit

  - What do you prefer? Implicit or Explicit?

Implicit contexts require little human/computer communication

Classification of context

- Type of Context
  - User Context, Temporal Context, Physical Context, Computing Context, Context History



Another classification of context



- Identity: user characteristics, needs
- Spatial: location, orientation, speed
- Temporal: date, time of day, season
- Environmental: temperature, light, noise
- Social: people nearby, activity, calendar
- Resources: nearby, availability, energy
- Computation: CPU, OS, memory, interfaces
- Network: wire/wireless, bandwidth, error rate
- Physiological: blood pressure, heart rate, tone of voice
- Psychological: preference, emotion, tiredness



Apps on your phone that change behaviour?



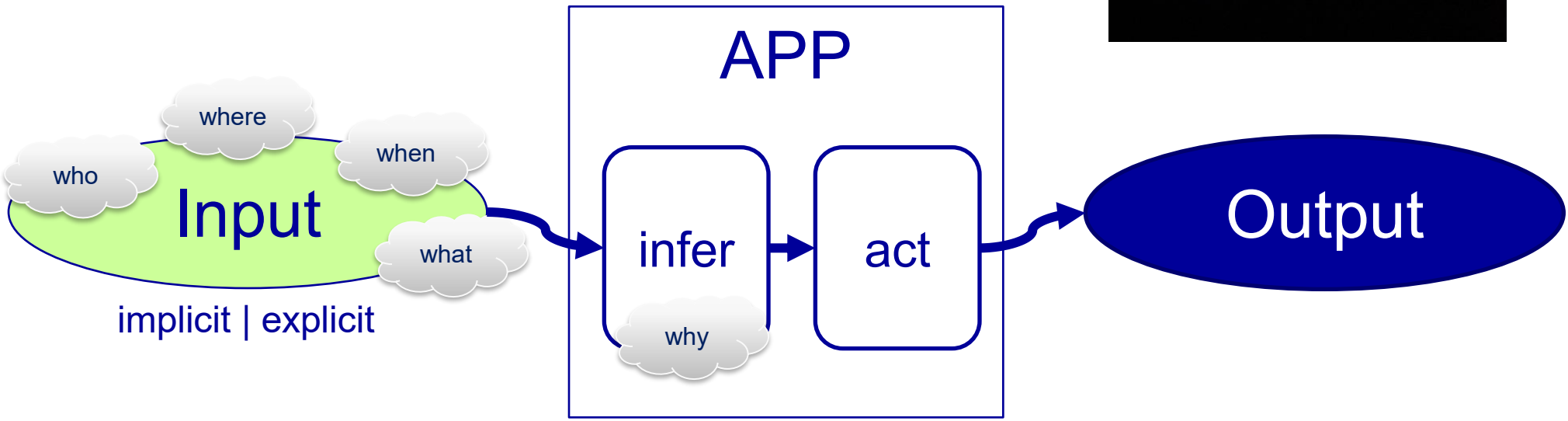
Design process of context-aware applications

1. **Specification** – determine what **context-aware behaviors** the application will have
2. Acquisition – determine what sensors are required to collect the contexts
3. Delivery – specify how context should be delivered from sensors to the applications
4. Reception – application “subscribes” to contexts that are of interest
5. **Action** – analyze the received context to determine how to execute it

Context-aware applications







Drunk User Interfaces

Drunk User Interfaces: Determining Blood Alcohol Level through Everyday Smartphone Tasks

Alex Mariakakis[1], Sayna Parsi[2,3], Shwetak N. Patel[1], Jacob O. Wobbrock[3]

[1]Computer Science & Engineering, [2]Human Centered Design & Engineering, [3]The Information School
DUB Group
University of Washington
Seattle, WA 98195 USA
{atml5, shwetak}@cs.washington.edu, {parsis, wobbrock}@uw.edu

ABSTRACT

Breathalyzers, the standard quantitative method for assessing inebriation, are primarily owned by law enforcement and used only after a potentially inebriated individual is caught driving. However, not everyone has access to such specialized hardware. We present *drunk user interfaces*: smartphone user interfaces that measure how alcohol affects a person's motor coordination and cognition using performance metrics and sensor data. We examine five drunk user interfaces and combine them to form the

are typically used after a drunk driver has been caught, rarely to prevent people from driving in the first place. Jewett *et al.* [14,23] estimate that the average drunk driver has driven drunk over 80 times before their first arrest. There remains a need of being able to catch cases of drunk driving without the presence of law enforcement or relying on people to determine their own limits for personal safety.

One can view inebriation as a temporary "situational impairment" that affects a person as they interact with the

Can you imagine what data can be used?

Can you imagine what data can be used?

- The phone estimates blood alcohol level only from everyday tasks:
  - **Typing:** speed, errors, corrections, pressure
  - **Swiping:** speed, accuracy and straightness of the gesture
  - **Balance:** body sway while holding the phone still (accelerometer and gyroscope)
  - **Reaction time:** simple and choice reaction tests

Can you imagine what data can be used?

- Beyond the paper, other context could be added:
  - **From a wearable:** heart rate and skin temperature
  - **From the phone:** gait (walking pattern), location (a bar at 2 a.m.), time of day and day of week, and app usage (dating or ride-sharing apps, number of messages sent)
  - **Other signals:** speech (slurred voice) and eye movement through the camera.

Can you imagine what actions can be done with that data?

Can you imagine what actions can be done with that data?

- Actions range from gentle to strict:
  - **Inform:** “You may be over the limit” warnings
  - **Suggest:** a taxi or ride-share, a call to a friend, or the way home
  - **Delay:** a “drunk mode” that holds messages or social media posts until the next morning, or asks “Are you sure?” before sending
  - **Protect:** blocking online purchases or gambling, or asking for a second authentication
  - **Prevent:** stopping car unlocking or start in connected cars, or alerting a chosen contact

- *What do you prefer?*

Discussion point

- Every action raises questions:
  - **False positives:** someone tired or with a motor impairment could be misjudged
  - **Who decides:** the user, or the system?
  - **Inform, alert or block:** what is the right level of intervention?
  - **Privacy:** who else could use this data, for example insurers, employers or the police?

Issues of context-aware applications

- Context is a proxy for human intent
- Context ambiguity or errors
- Rules vs machine learning
- Privacy
- Evaluation
- End-user specific issues

Issues of context-aware applications

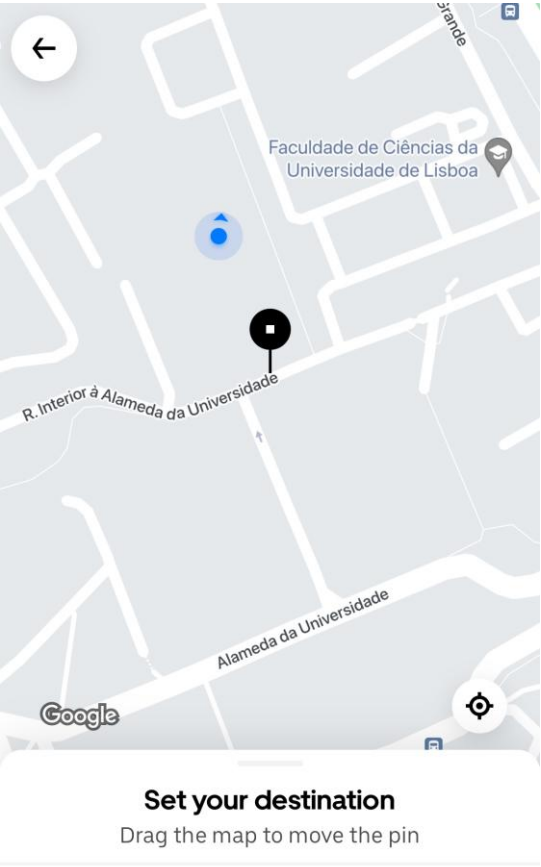
- Context is a **proxy for human intent**

- Ideally, context awareness completely understands human intent to adapt the application



Issues of context-aware applications

- Context ambiguity or errors
  - Sensors can sense incorrectly, fail
  - Inferencing systems can reach inaccurate conclusions
  - Model ambiguity as part of the context
    Often as a likelihood or certainty of a particular context
  - Combine multiple sources of context to increase accuracy
    sensor fusion
  - Allow users to manually disambiguate



Rules vs machine learning

- Context aware often built from If/Then type rules
  - e.g. If I am home and it is night, turn on the lights
- Advantages:
  - Easy to define
  - Easy to evaluate
  - Users can understand (Mental Model)
- Disadvantages:
  - Complex dependencies can create conflicts
  - Difficult to debug when the number of rules is large
  - Can be inefficient to evaluate all rules

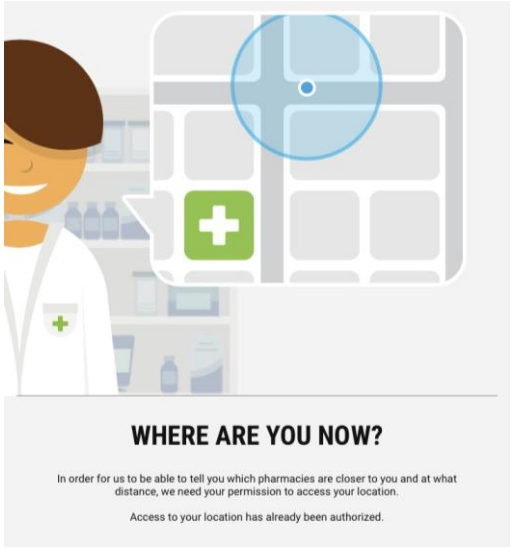
Rules vs machine learning

- Gather data on situations where adaptation is desired
- Use machine learning to apply probabilistic relationships
- Advantages:
  - Skip context-inferencing step – ML learns the contexts
  - Not static, can improve recommendations
- Disadvantages:
  - Users may not understand rules (Mental Model)
  - Large amount of data to learn
  - Requires infrastructure for processing

Privacy

Principles

- Context-aware systems often gather a large amount of data
- Context information must be shared only among components that need it
- Appropriate steps need to be taken to prevent improper or malicious dissemination of these data
- User's privacy should be maintained



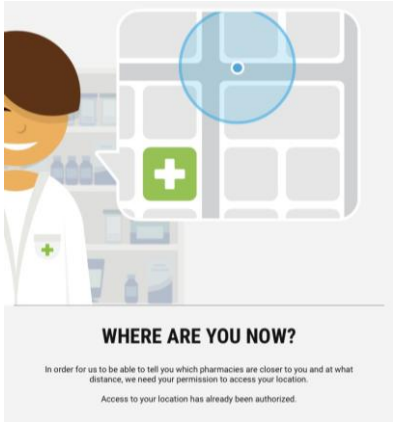
Privacy

Principles

- Context-aware systems often gather a large amount of data
- Context information must be shared only among components that need it
- Appropriate steps need to be taken to prevent improper or malicious dissemination of these data
- User's privacy should be maintained

In practice today

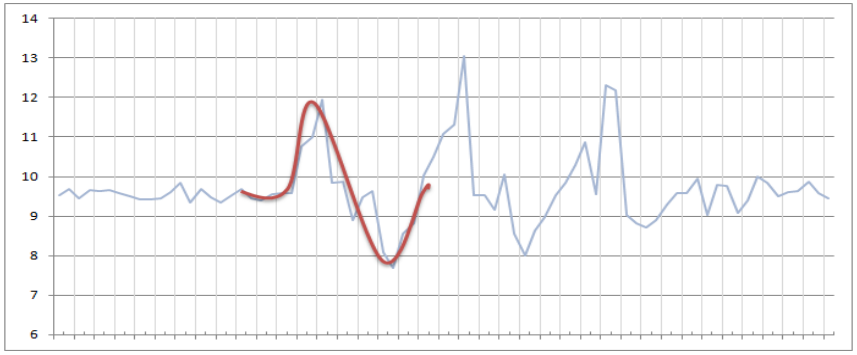
- **Granular permissions:** approximate vs precise location, "only while using the app", alerts when an app uses location in the background
- **Usage indicators:** camera and microphone indicators show when these sensors are active
- **Tracking:** App Tracking Transparency (iOS): apps must ask before tracking users across other apps and websites
- **GDPR:** data minimisation and informed, explicit consent
- **On-device processing:** infer context on the phone and send only what is needed, if anything

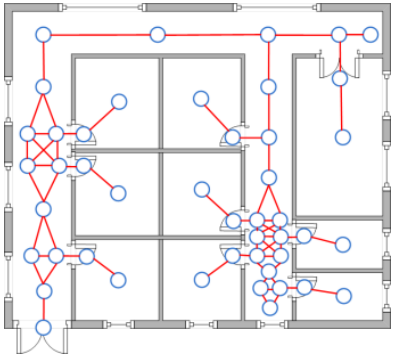


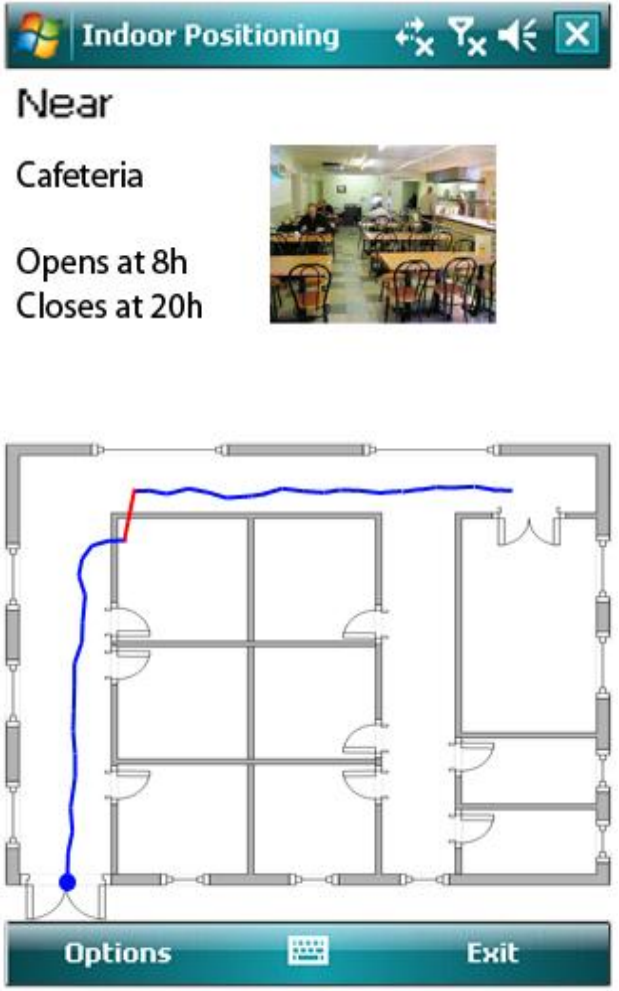
Issues of context-aware applications

- Evaluation
  - Context-aware apps rarely can be tested in a lab environment
  - Even in the field some situations might be challenging to evaluate

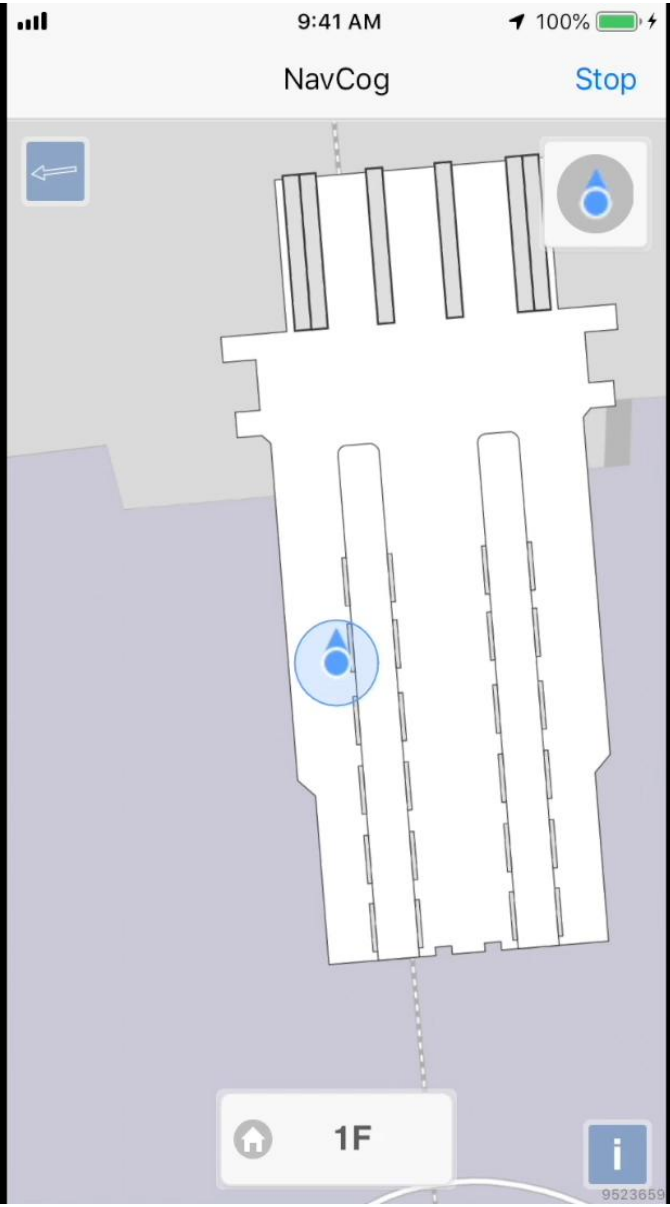
PoI Awareness Using Indoor Positioning with a Mobile Phone.







NavCog3 @Pittsburgh International Airport



End-user specific issues

- Understanding – mental model
  - how a user forms an understanding of a context-aware application's behavior
  - Context-aware developers need to make sure users know why contextual adaptations are made
    - Present the context
    - Use human-intelligible rules
- User control
  - Control vs. One-size-fits-all
  - Users often prefer to be able to adapt systems to their usage patterns
  - Allow evolving usage patterns based on users' preferences

Future Trends – Now what?

- AI Assistants / Companions
- Context Aware AR/VR
- Healthcare Wearables with predictive alerts
- Smart Cities